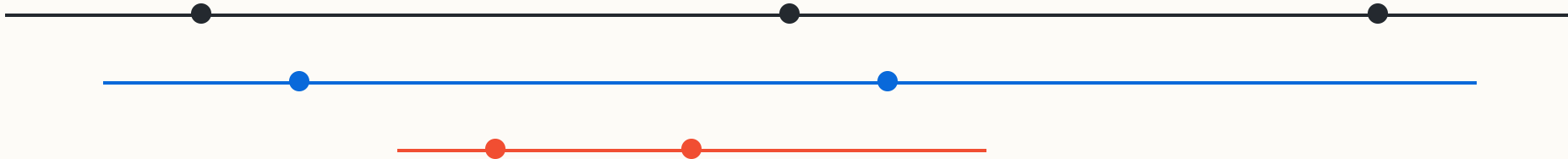


Gitflow: ramas, commits y colaboración

Una guía visual para entender el modelo Gitflow y trabajar con cambios controlados.



¿Por qué existen las ramas?

Las ramas son líneas temporales paralelas que permiten a los desarrolladores trabajar sin interferir en el código estable.

Sin ramas, cada cambio afectaría directamente a producción, haciendo imposible la colaboración en equipo y la experimentación segura.



Aislamiento

Protege el código estable de errores nuevos.



Paralelismo

Varios desarrolladores, varias tareas a la vez.



Experimentación

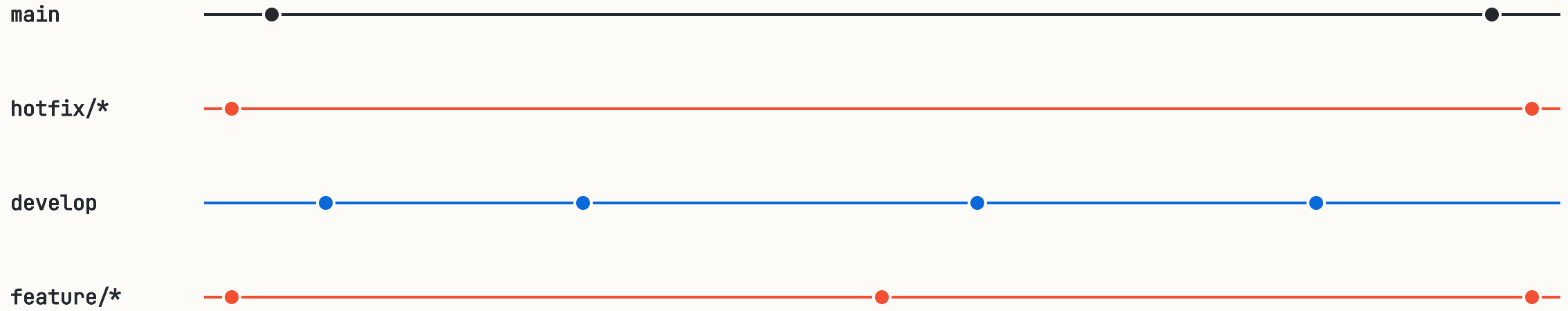
Prueba ideas sin miedo a romper nada.



Trazabilidad

Historial limpio y organizado por tareas.

La anatomía de Gitflow



Gitflow organiza el trabajo en ramas con propósitos específicos: permanentes (main, develop) y temporales (feature, release, hotfix).

main y develop: las ramas permanentes

main

- **Producción:** Contiene el código que está en manos del usuario final.
- **Sagrada:** Nunca se hace commit directo aquí.
- **Etiquetada:** Cada merge a main genera una nueva versión (Tag).

develop

- **Integración:** Donde se unen todas las nuevas funcionalidades.
- **Base:** Es la rama de partida para cualquier nueva tarea.
- **Pre-producción:** Refleja el estado del próximo lanzamiento.

HEAD → gitflow / feature

Feature branches: una tarea, una rama

01

Crear desde develop

Asegúrate de tener la última versión de develop antes de empezar.

02

Trabajar y Commitear

Realiza cambios atómicos relacionados solo con esa funcionalidad.

03

Pull Request / Merge

Una vez terminada, la rama se integra de vuelta a develop y se borra.

```
# Crear nueva funcionalidad
git checkout develop
git pull origin develop
git checkout -b feature/login-social

# Finalizar e integrar
git checkout develop
git merge --no-ff feature/login-social
git branch -d feature/login-social
```

Commits: unidades de progreso



Atómicos: Un commit debe resolver una sola cosa. Si haces dos tareas, haz dos commits.



Descriptivos: El mensaje debe explicar el "por qué", no solo el "qué".



Funcionales: Nunca subas código que rompa la compilación o los tests.

- [a7f2c1](#) feat: add social login buttons
- [8e3d4b](#) fix: layout alignment on mobile
- [5c1a9f](#) docs: update readme with API keys

Release y Hotfix: preparar y reparar

release/*

- **Planificada:** Se crea cuando develop tiene todas las features del próximo lanzamiento.
- **Estabilización:** Solo se permiten correcciones de errores finales.
- **Destino:** Se mezcla en main (producción) y en develop.

hotfix/*

- **Urgente:** Se crea directamente desde main para corregir un error crítico en producción.
- **Rápida:** Evita esperar al ciclo normal de desarrollo.
- **Destino:** Se mezcla en main y en develop inmediatamente.

Gitflow de principio a fin



Reglas para que Gitflow funcione



Nomenclatura clara

Usa prefijos: feature/, bugfix/, hotfix/. Sé consistente.



Actualiza a menudo

Haz `git pull origin develop` frecuentemente para evitar conflictos grandes.



Revisión de código

Nunca hagas merge a develop sin que otro compañero revise tu Pull Request.



Limpieza

Borra las ramas locales y remotas una vez que el merge se haya completado.

HEAD → gitflow / cierre



¡Listo para despegar!

```
git push origin main --tags
```